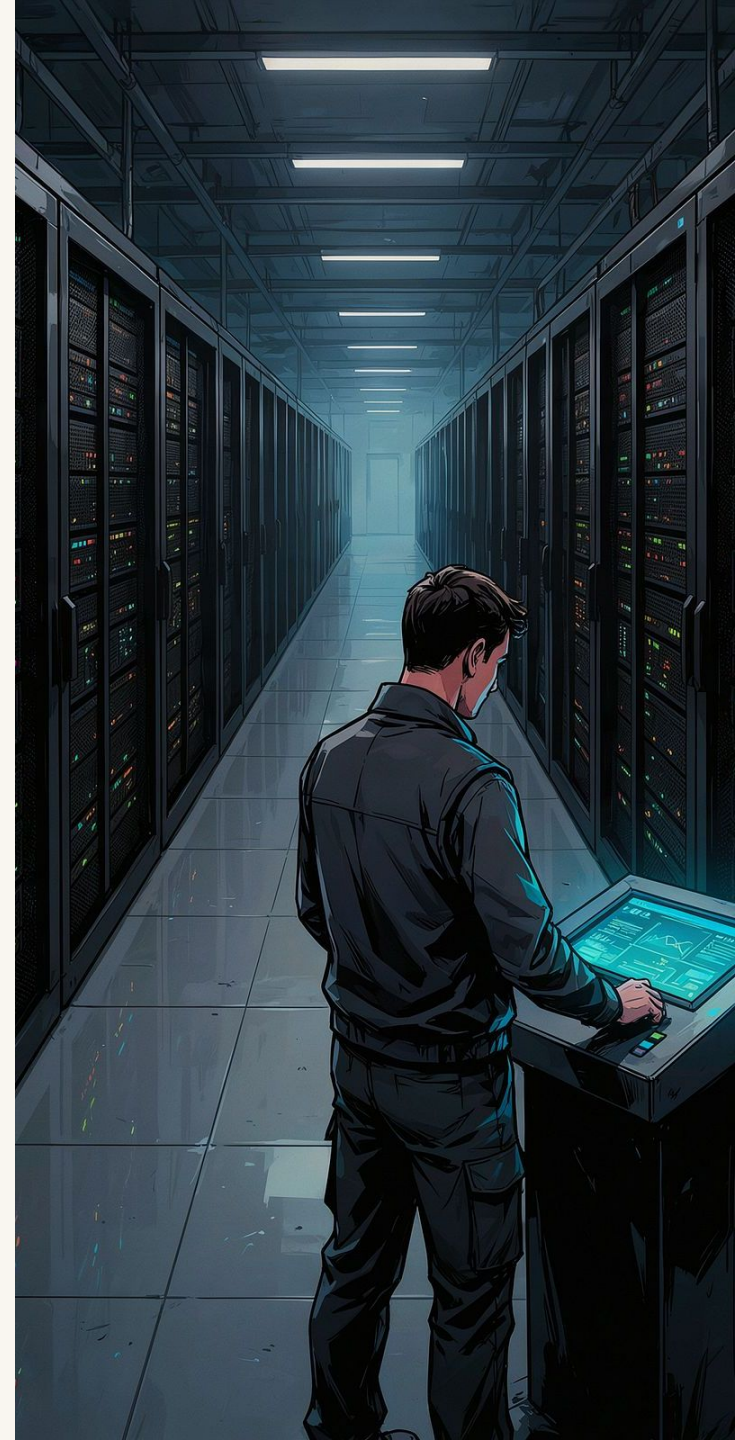


MODULE 8 — Harness CI/CD Setup & Architecture

From Jenkins Controller/Agent to Harness Platform/Delegate

Understanding Harness through a Jenkins Engineer's Perspective — a structured walkthrough of deployment models, delegate architecture, and CI build infrastructure.

❌ Do NOT assume Harness = Jenkins running in another container. This distinction is the most critical concept in this module.



Learning Objectives

By the end of this module, you will be able to confidently distinguish between Harness deployment models, understand the role of the Delegate, and map familiar Jenkins concepts to their Harness equivalents.

Deployment Models

- Harness SaaS
- Harness Delegate
- Self-Managed Enterprise
- Harness Open Source

Delegate Types

- Docker Delegate
- Kubernetes Delegate
- Delegate vs. Platform
- Delegate vs. Jenkins Agent

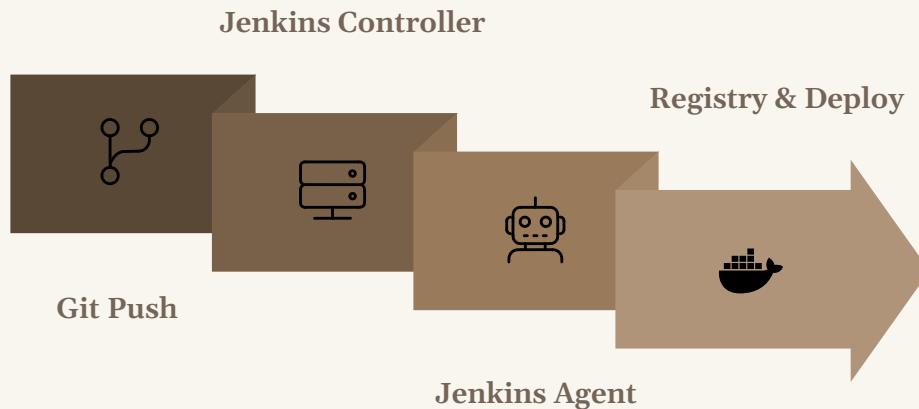
Integrations

- Harness + EKS Anywhere
- Harness + ArgoCD
- CI Build Infrastructure
- Jenkins Equivalents

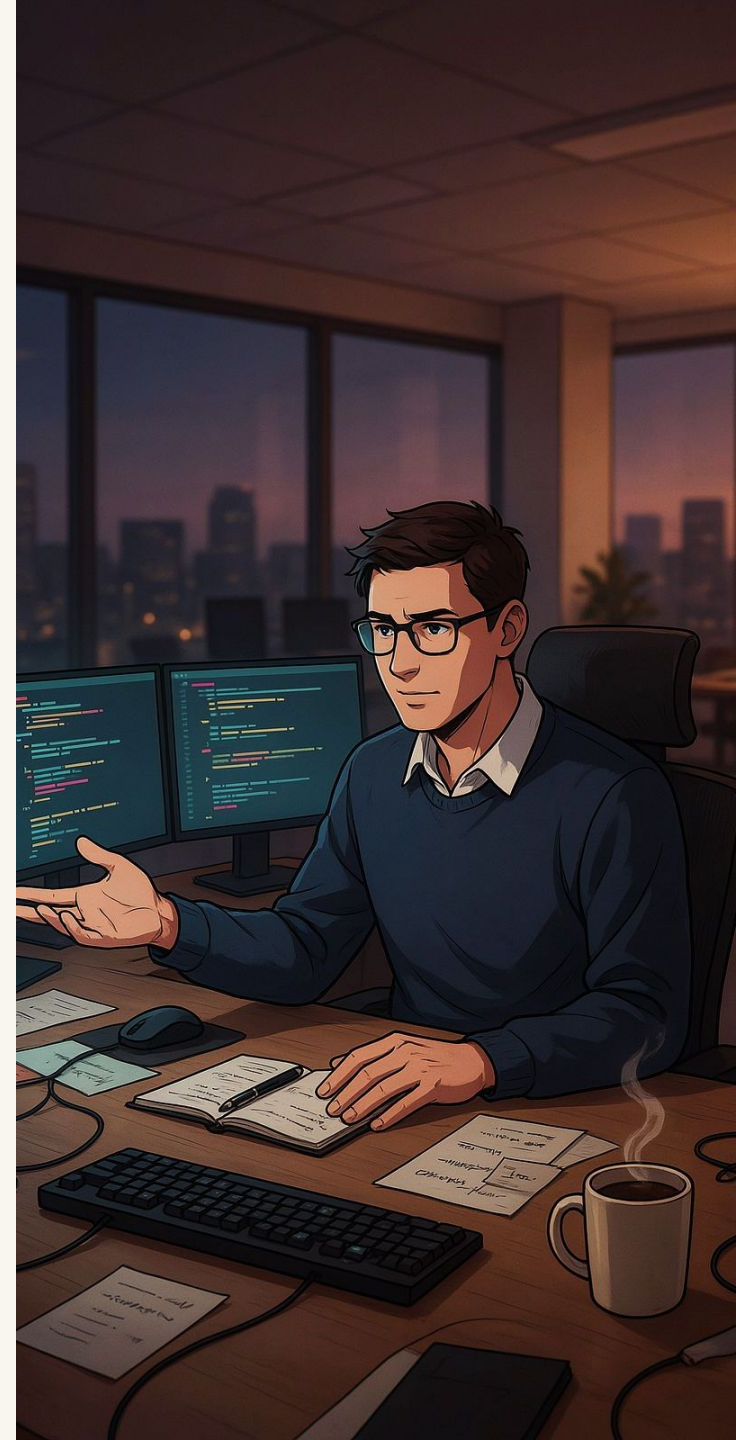
Start With What You Already Know

The Jenkins Architecture You've Worked With

Before introducing Harness, let's anchor to the Jenkins mental model you already understand. Every Harness concept will be compared back to this baseline.



This is the complete Jenkins pipeline loop — from developer commit to deployed artifact. Keep this picture in mind as we map it to Harness.



Jenkins Controller: What It Actually Does

Jenkins Controller Responsibilities

- Hosts the web UI and dashboard
- Orchestrates pipeline execution
- Stores and manages credentials
- Manages job and pipeline configuration
- Handles plugin installation and lifecycle
- Schedules builds and distributes work

Jenkins Agent Responsibilities

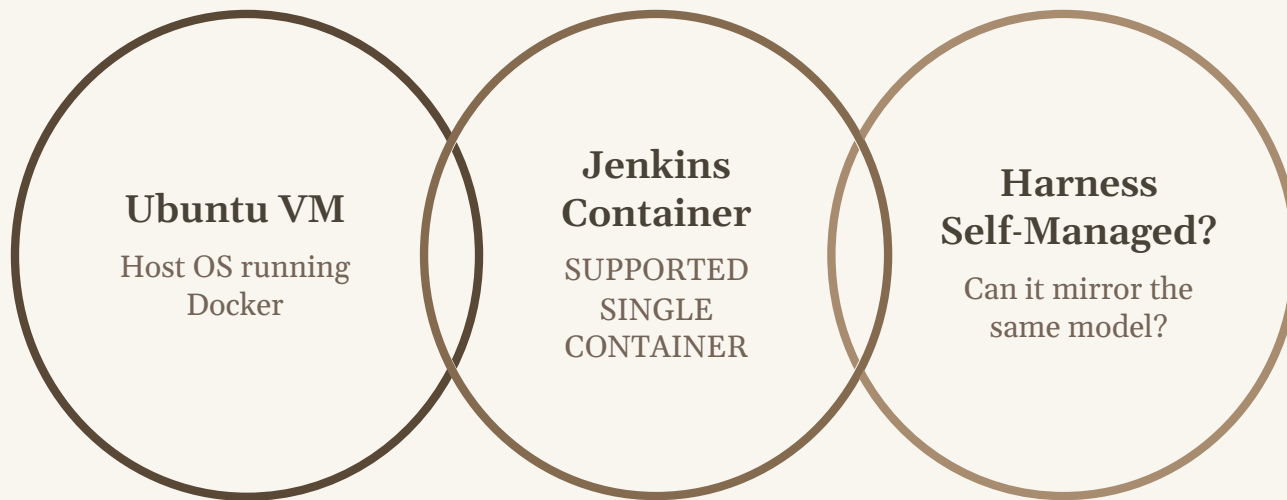
- Executes shell commands and scripts
- Runs build and compile steps
- Executes automated tests
- Builds and tags Docker images
- Pushes artifacts to registries
- Applies Kubernetes manifests

The Controller **thinks**. The Agent **acts**. This separation matters greatly when mapping to Harness.

Jenkins Runs in Docker — So Can Harness, Right?

The Question Every Student Asks

You've already seen Jenkins packaged as a Docker container. It's a single, self-contained image. This leads to a very natural — but critically incorrect — assumption about Harness.



✗ NOT THE SAME DEPLOYMENT MODEL — Harness Self-Managed Enterprise Edition is a multi-service Kubernetes platform, not a single Docker container. This is the most important distinction in this entire module.

Harness Deployment Models Overview

Harness is not one thing — it is a family of products with distinct deployment models. Each model serves a different use case, has different infrastructure requirements, and maps differently to what you know from Jenkins.



Harness Open Source is a **separate product** from Harness Self-Managed Enterprise Edition. Do not conflate them. Each branch of this diagram has its own installation method and operational model.

Option 1 — Harness SaaS

Harness Manages the Control Plane — You Don't

With Harness SaaS, the entire Harness platform — UI, pipeline engine, secret management, connectors — is hosted and maintained by Harness. You interact with it over the internet.

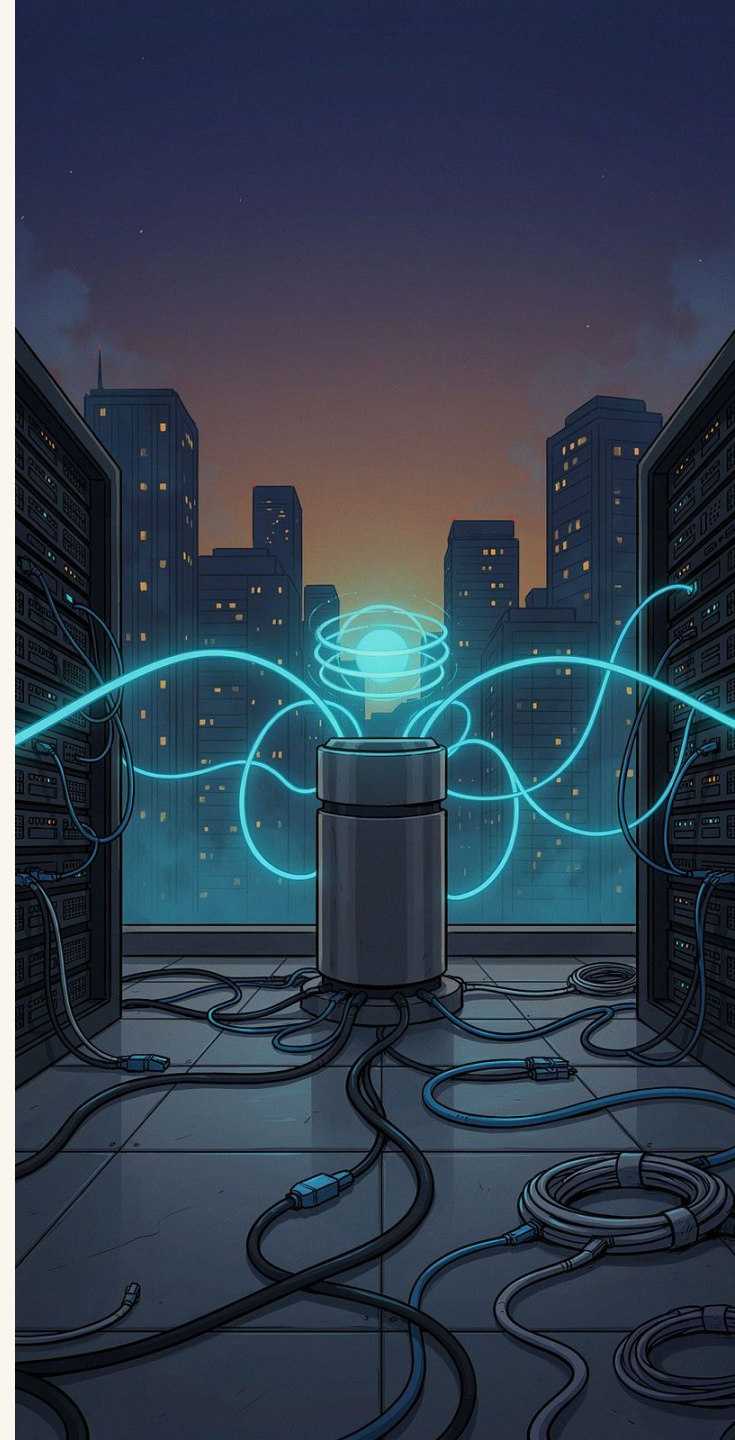
Advantages

- Fast setup — no platform installation
- No database or upgrade management
- No Kubernetes cluster required for the platform
- Ideal for learning and enterprise SaaS adoption
- Harness handles availability and scaling

Disadvantages

- Control plane lives outside your infrastructure
- Requires outbound internet connectivity
- May not meet compliance or air-gap requirements
- Less control over upgrade timing

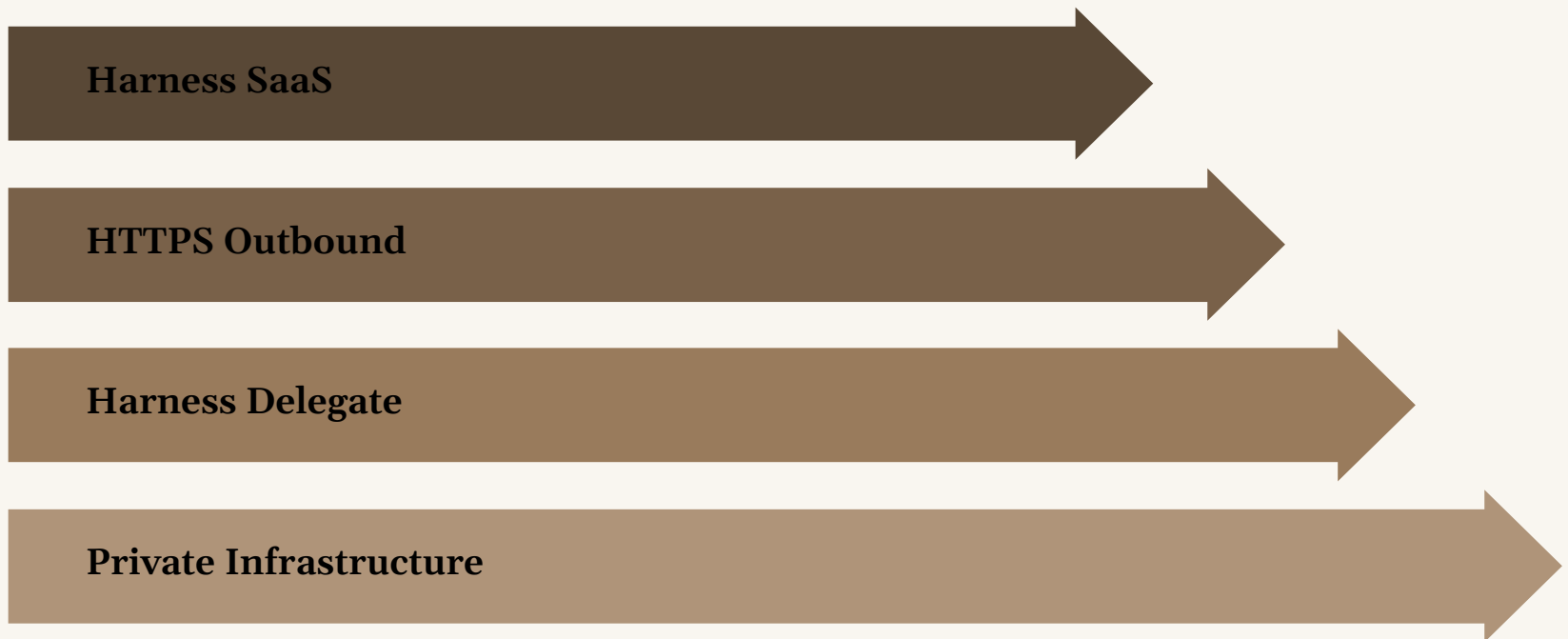
Jenkins comparison: With Jenkins, you install and manage the Controller. With Harness SaaS, Harness *is* the Controller — hosted for you.



Option 2 — The Harness Delegate

The Most Misunderstood Component

The Delegate is **not** the Harness platform. It is a lightweight worker agent that you deploy inside your own infrastructure. It reaches out to Harness SaaS over HTTPS and executes tasks on behalf of the platform.



- ❏ Conceptually similar to a Jenkins Agent — but not technically identical. The Delegate initiates the connection outbound to Harness. Jenkins Agents are connected inbound from the Controller.

Jenkins Agent vs. Harness Delegate

Understanding the conceptual mapping — and the important differences — prevents architectural mistakes when designing your Harness pipelines.

Concept	Jenkins	Harness
Execution worker	Jenkins Agent	Harness Delegate
Connection direction	Controller → Agent (inbound)	Delegate → SaaS (outbound)
Deployment location	VM, Pod, container	Docker, Kubernetes Pod
Manages credentials?	Via Controller	Fetches from Harness Secrets
Required for SaaS?	N/A — Controller is self-hosted	YES — Delegate required for private infra

Key insight: The Delegate is conceptually similar in role, but architecturally different. It is not a passive listener — it actively polls Harness SaaS for tasks.

Delegate Running in Docker

Harness Delegate in Docker $\neq$ Harness Platform in Docker

Harness fully supports installing the Delegate as a Docker container on a VM. This is a legitimate, supported deployment method — but it is critical to understand what it represents.



What This IS

- Harness Delegate running as a container
- Supported installation method
- Good for VM-based lab environments
- Lightweight — single container

What This IS NOT

- Not the Harness control plane
- Not Harness Self-Managed Enterprise
- Not equivalent to running Jenkins Controller in Docker
- Does not host the Harness UI or pipeline engine

Delegate Running in Kubernetes

The Preferred Cloud-Native Approach

Deploying the Harness Delegate as a Kubernetes Pod inside your EKS Anywhere cluster is the most elegant and Kubernetes-native approach. The Delegate Pod sits close to the Kubernetes API and can execute manifests, Helm charts, and pipeline tasks without exposing the API externally.



Harness SaaS

HTTPS Outbound

EKS Anywhere

Delegate Pod



Kubernetes-Native

Deployed using Helm or YAML manifests. Managed with standard Kubernetes tooling — same as your ArgoCD and Prometheus deployments.



Private by Design

The Delegate initiates outbound connections only. No inbound firewall rules or public exposure of your Kubernetes API is required.



Best Fit for Training

Ideal for your EKS Anywhere training cluster. Jenkins comparison: a Jenkins Agent Pod running inside the same cluster.

What Can We Actually Do With Docker?

A Clear YES / NO Reference Table

Use this table whenever there is ambiguity about what Docker-based Harness deployments are valid, supported, and recommended.

Action / Question	Answer
Run Harness Delegate as a Docker container?	✓ YES — Supported
Use Docker Delegate to reach private infrastructure?	✓ YES — Supported
Use Docker Delegate to communicate with Harness SaaS?	✓ YES — Supported
Run Harness Open Source using Docker?	✓ YES — Supported (separate product)
Run complete Harness Self-Managed Enterprise as one Docker container?	✗ NO — Not the deployment model
Treat Docker Delegate as the Harness control plane?	✗ NO — Delegate ≠ Platform
Install full Harness SMP with docker-compose?	⚠ NOT THE SUPPORTED SMP MODEL

📌 Delegate ≠ Harness Platform. This distinction must be internalized before moving forward.

Option 3 — Harness Self-Managed Enterprise Edition

The Full Control Plane, Inside Your Infrastructure

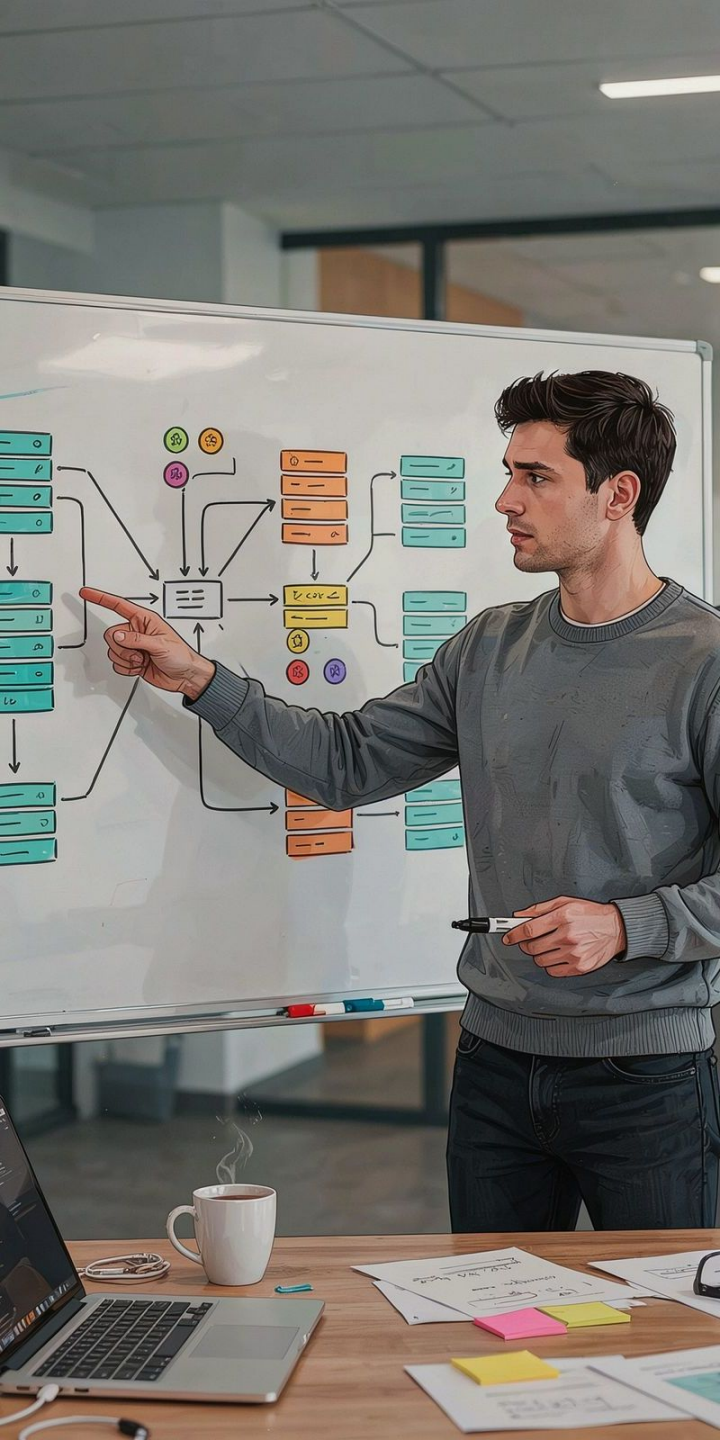
Harness Self-Managed Enterprise Edition (SMP) allows organizations to run the entire Harness control plane — UI, pipeline engine, secrets, governance, CI, CD — entirely within their own data center or private cloud. The organization is responsible for operating the underlying platform.

Your Organization Operates

- Kubernetes cluster (the runtime platform)
- Networking, TLS, and DNS configuration
- Persistent storage and database architecture
- Backup, DR, and upgrade management
- Monitoring and alerting for platform health

Harness Provides

- Platform software via Helm charts
- Versioned releases and upgrade paths
- Support and documentation
- License and entitlement management



Self-Managed Is NOT One Container

The Most Critical Architectural Truth in This Module

The instinct to look for `docker run harness-controller` is natural — but incorrect. Harness Self-Managed Enterprise Edition is a distributed, multi-service platform that requires Kubernetes as its runtime substrate.

✗ Incorrect Mental Model

~~`docker run harness/harness-controller`~~

There is no single Harness container that behaves like a Jenkins Controller. This image does not exist for Self-Managed Enterprise.

✓ Correct Architecture

- Kubernetes Cluster (required runtime)
- Harness platform services (CI, CD, pipelines)
- Supporting services (Redis, MinIO, etc.)
- Database components (MongoDB / TimescaleDB)
- Ingress and networking layer
- Installed via Helm charts

⊗ Do not teach or demonstrate docker-compose as a path to Harness Self-Managed Enterprise Edition. It is not the supported installation model.

Harness Self-Managed — Installation Model

Kubernetes + Helm Is the Supported Path

Just as you installed ArgoCD, Prometheus, and Grafana into your EKS Anywhere cluster using Helm, Harness Self-Managed Enterprise Edition follows the same pattern — but at significantly larger scale.



The Helm chart deploys dozens of microservices into your cluster. This is why SMP requires a dedicated cluster — it is not a lightweight add-on. Think of it as installing an entire platform, not a single application.

Jenkins Self-Hosted vs. Harness Self-Managed

Both allow you to host the CI/CD control plane inside your own infrastructure. But the architectural weight and complexity are in entirely different leagues.

Component	Jenkins Self-Hosted	Harness Self-Managed
Control plane	Jenkins Controller	Harness Platform (multi-service)
Execution worker	Jenkins Agent	Harness Delegate
Secret storage	Jenkins Credentials	Harness Secrets Manager
Extensibility	Jenkins Plugins	Connectors / Integrations
Pipeline definition	Jenkinsfile (Groovy)	Harness Pipeline (YAML)
Reusable logic	Shared Libraries	Harness Templates
Installation method	WAR / Docker / Package	Kubernetes + Helm
Runtime requirement	JVM (single process)	Kubernetes cluster (required)

Similar purpose in some areas. Fundamentally different architecture.

Can Harness Self-Managed Be Installed Like ArgoCD?

Yes — Conceptually. But Scale Is Dramatically Different.

ArgoCD is installed into Kubernetes using Helm or kubectl manifests. Harness Self-Managed Enterprise Edition follows the same pattern. But the comparison ends at the installation method.

ArgoCD in Kubernetes

- GitOps CD platform
- Kubernetes-native, small footprint
- Focused: deployment and GitOps sync
- Few services, low resource requirement
- `helm install argocd ...`

Harness SMP in Kubernetes

- Full CI/CD platform with multiple modules
- CI, CD, Pipelines, Secrets, Governance
- Dozens of microservices and supporting components
- Requires significant CPU, memory, and storage
- Helm-based installation, dedicated cluster recommended

⚠ Installing Harness SMP inside Kubernetes is architecturally valid, but it is dramatically heavier than ArgoCD. Never co-locate it with student workloads on a shared cluster without careful capacity planning.

ArgoCD vs. Harness Platform — Scope Comparison

ArgoCD

- GitOps-focused CD tool
- Kubernetes-native by design
- Lightweight, minimal resource footprint
- Sync manifests from Git to cluster
- Strong in: deployment, drift detection
- Limited in: CI, secrets, governance

Harness SMP

- Full CI/CD enterprise platform
- CI pipelines and build infrastructure
- CD with deployment strategies
- Secrets management module
- Connectors and integrations catalog
- Templates, governance, policy enforcement
- Feature Flags, CCM, Chaos modules

Harness can **replace or complement** ArgoCD depending on your architecture. In this training, we use Harness SaaS + ArgoCD together — a common real-world pattern.

Resource Collision and Operational Risk

Existing Student Cluster

- ArgoCD
- Prometheus + Grafana
- Loki log aggregation
- MetalLB load balancing
- NGINX Ingress
- Student applications

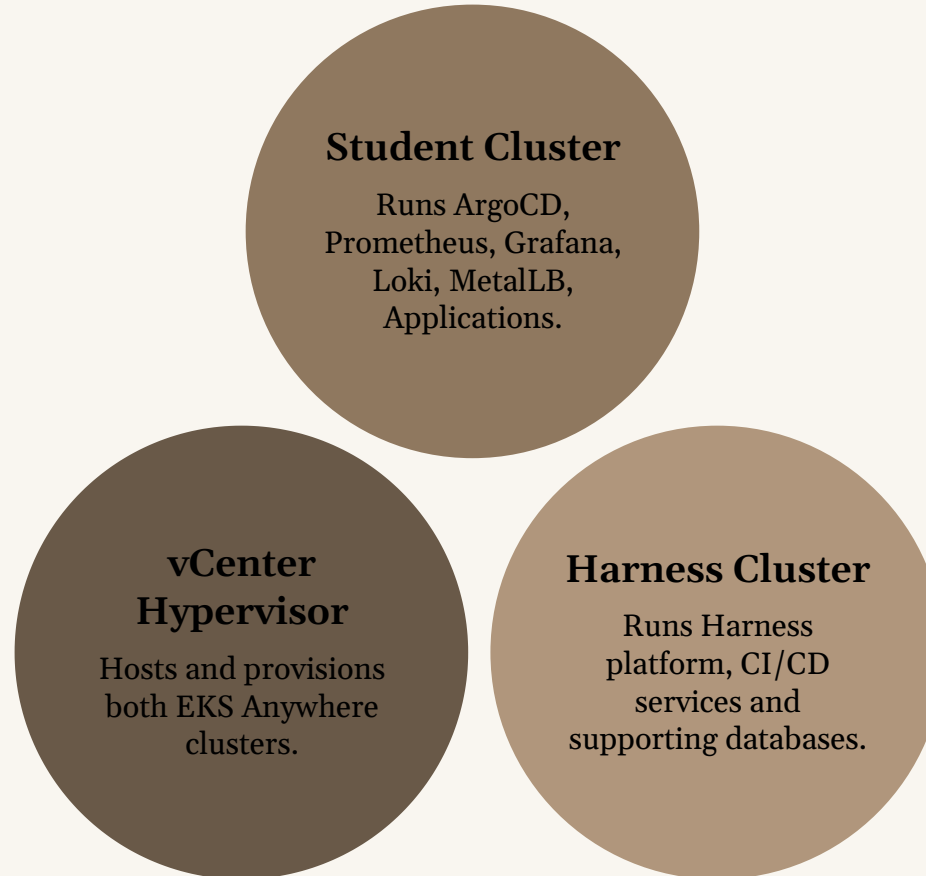
- High CPU/memory consumption from platform microservices
- Significant persistent storage requirements (databases)
- Upgrade complexity and potential downtime
- Database management overhead (MongoDB, etc.)
- Failure blast radius affects ALL student workloads
- Operational complexity beyond training scope

⚠ Do not casually install Harness SMP into the main student workload cluster. Use a dedicated cluster or Harness SaaS for training.

Recommended Training Architecture

Two Clusters — Clear Separation of Concerns

The safest and most instructive training architecture separates student workloads from Harness platform workloads using two distinct EKS Anywhere clusters within your vCenter environment.

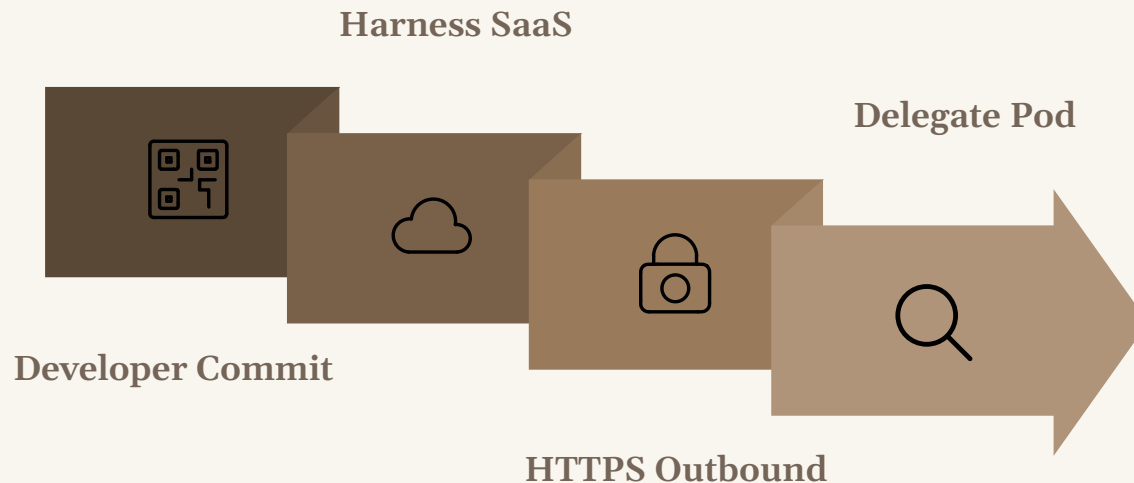


This separation ensures that Harness platform upgrades, restarts, or failures do not impact student application workloads. Each cluster can be sized independently and managed by different teams if needed.

Harness SaaS + EKS Anywhere

The Recommended Lightweight Training Architecture

For most of this training, you will use Harness SaaS as the control plane and deploy a Harness Delegate into your EKS Anywhere cluster. This avoids all platform installation complexity while teaching real Harness concepts.



✓ No Platform Install

Zero Harness infrastructure to manage

✓ Private Cluster

EKS Anywhere stays private behind the Delegate

✓ Low Resources

Only the Delegate Pod runs in your cluster

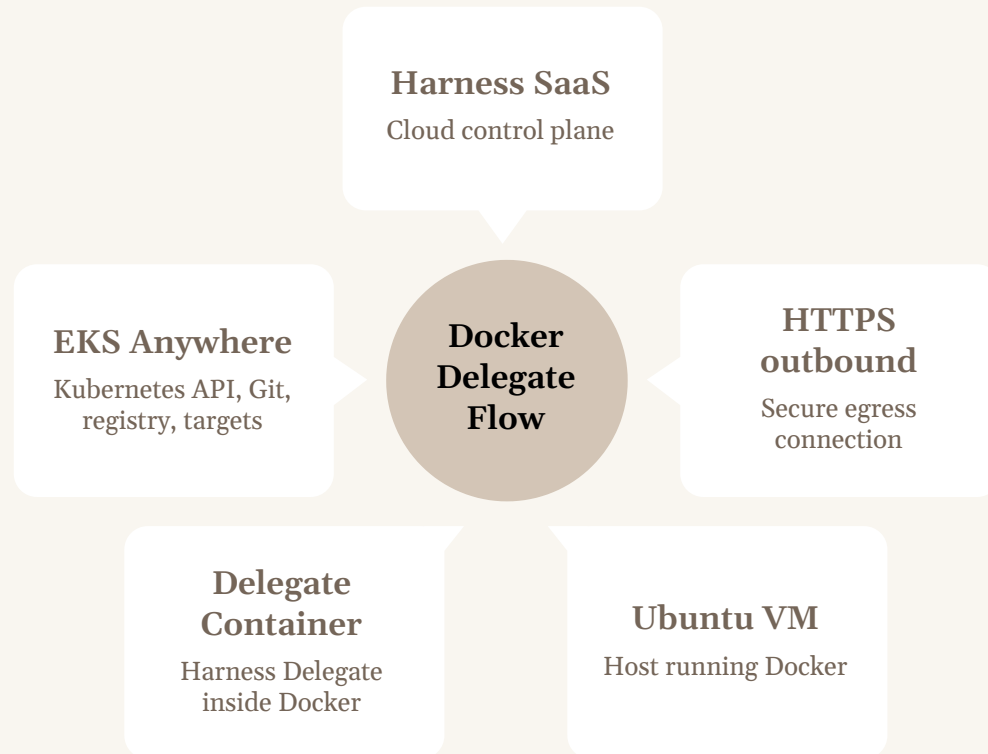
✓ Full Learning

All Harness concepts accessible via SaaS

Harness SaaS + Docker Delegate

The Closest Equivalent to Jenkins Controller + Agent — Without Self-Hosting

If you want to demonstrate how a private VM connects Harness SaaS to your EKS Anywhere cluster using a familiar Docker-based model, the Docker Delegate architecture is your answer.



Jenkins Parallel

This architecture most closely mirrors:

Jenkins Controller (SaaS) → Jenkins Agent (Docker on VM)

The Harness SaaS platform plays the Controller role. The Docker Delegate plays the Agent role. The key difference: the Delegate initiates outbound — the Agent does not.

Use Cases

- Demonstrating VM-based private connectivity
- Environments without Kubernetes for the Delegate
- Quick Delegate registration for labs
- Showing Delegate flexibility to students

Harness SaaS + Kubernetes Delegate

The Preferred Kubernetes-Native Training Path

After demonstrating the Docker Delegate, advance students to the Kubernetes Delegate model. This is the preferred approach for teams operating in cloud-native environments and for your EKS Anywhere training cluster.



Harness SaaS

HTTPS

EKS Anywhere

Delegate Pod



Installed via Helm

Deploy the Delegate into your cluster with a single Helm command. It registers automatically with your Harness account over outbound HTTPS.



Full Cluster Access

The Delegate Pod has kubectl and Helm available, enabling it to apply manifests, run Helm upgrades, and interact with any namespace in the cluster.



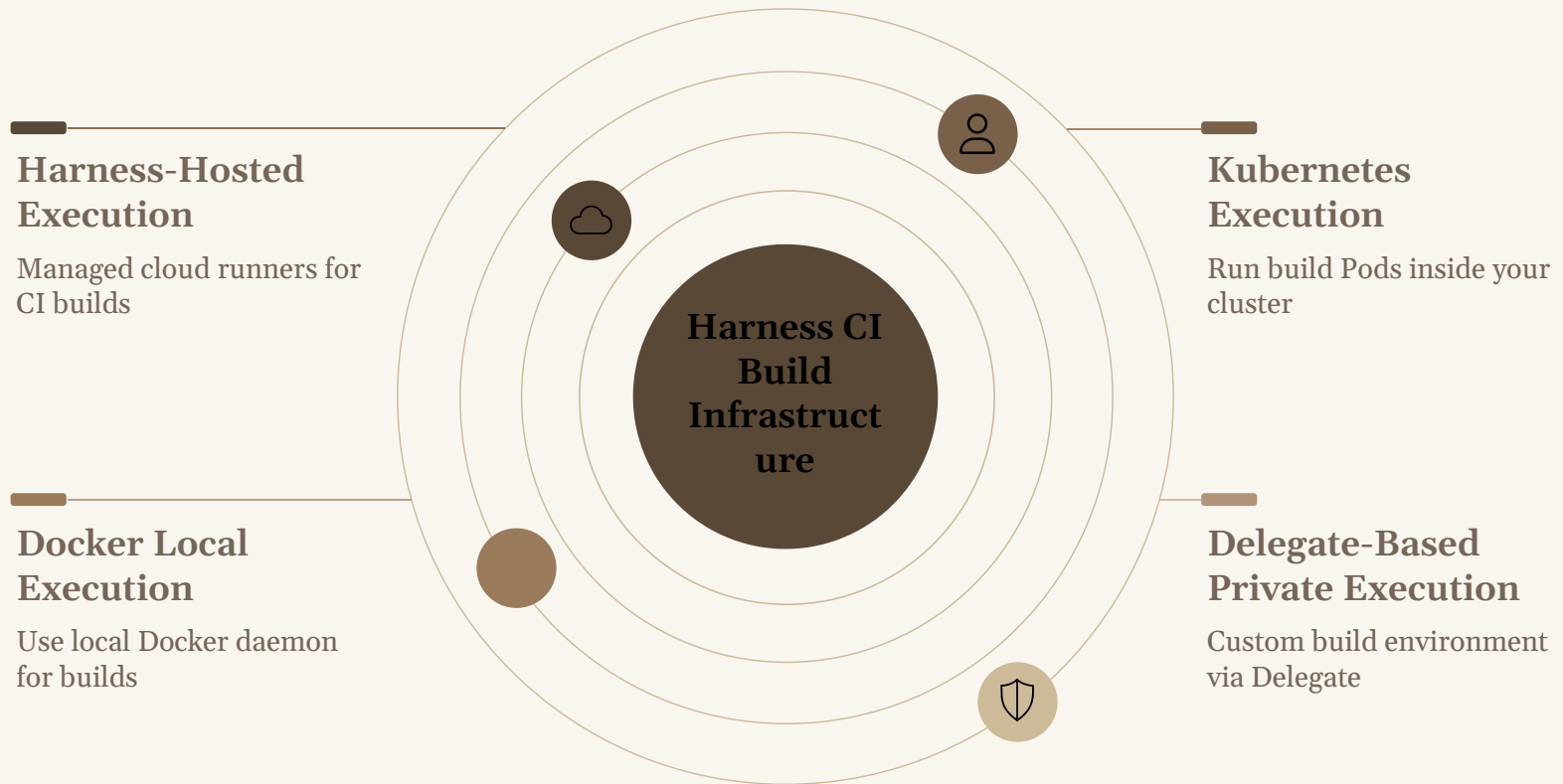
Learn This After Docker Delegate

Introduce the Kubernetes Delegate as the production-grade progression from the simpler Docker Delegate lab demonstration.

Harness CI Build Infrastructure

Where the Build Runs Is Separate From Where the Platform Runs

One of the most nuanced concepts in Harness architecture is that the **Harness control plane**, the **Harness Delegate**, and the **CI build environment** are three separate, independent architectural decisions.



You can run Harness pipelines using Harness-hosted runners (like GitHub Actions' hosted runners), your own Kubernetes cluster for build Pods, a local Docker daemon, or a custom private execution environment via the Delegate. These are independent choices per pipeline stage.

CI Build Infrastructure — Decision Matrix

Build Infrastructure	Where It Runs	Who Manages It	Best For
Harness-Hosted	Harness cloud	Harness	Quick start, SaaS teams
Kubernetes Execution	Your K8s cluster	You	Cloud-native, scalable builds
Docker / Local	Local Docker daemon	You	Simple labs, dev machines
Delegate-Based Private	Private infra via Delegate	You	Air-gapped, on-prem builds

Jenkins comparison: This mirrors the choice between Jenkins Cloud agents (ephemeral Pods), Docker agents, and static VM agents — but with cleaner separation of configuration from execution.

Harness Open Source

A Separate Product — Not Self-Managed Enterprise Edition

Harness Open Source is a community-oriented, open-source CI/CD product that Harness maintains separately from its enterprise product line. It has its own Docker-based quick-start model, which often causes confusion with Harness SMP.

Harness Open Source

- Open-source, community product
- Docker-based quick start exists
- Smaller scope and feature set
- Good for personal or small team use
- Separate codebase from Enterprise

Harness Self-Managed Enterprise

- Enterprise, commercially licensed
- Kubernetes + Helm installation only
- Full module suite (CI, CD, Governance, etc.)
- Designed for large organizations
- Support, SLAs, and enterprise features

⊗ Do not tell students to run Harness SMP using docker-compose. Use the supported deployment model for each specific Harness product. Harness Open Source ≠ Harness Self-Managed Enterprise Edition.

Docker Compose — What Is and Isn't Possible

A Definitive Reference for the Lab Environment

Docker Compose is a useful tool, but it is not a universal installation method for Harness. Use this table as a reference when students ask "can we just use docker-compose for this?"

Capability	Docker Compose Answer
Jenkins Controller	✓ YES — supported and common
Jenkins Agent	✓ YES — supported
Harness Delegate	✓ YES — supported installation method
Harness Open Source quick start	✓ YES — supported for Open Source product
Full Harness Self-Managed Enterprise	✗ NO — not the supported model
ArgoCD (for testing only)	⚠ POSSIBLE — but Kubernetes is the standard
Harness Kubernetes Delegate	✗ NO — use Kubernetes deployment manifests
Full Harness enterprise as one container	✗ NO — does not exist as a deployment model

Key message: Docker Compose is a tool for specific components — not a generic Harness installation method.

Harness + ArgoCD — Complementary Architecture

Two Tools, One Pipeline, Shared Kubernetes Cluster

In real-world and training environments, Harness and ArgoCD are often used together. Harness handles CI (build, test, push) and pipeline orchestration. ArgoCD handles GitOps-based CD (sync from Git to cluster). The Harness Delegate enables communication between both systems.



- This pattern is common in mature GitOps organizations. Harness owns the build and orchestration layer; ArgoCD owns the deployment and drift-reconciliation layer. Both coexist on the same EKS Anywhere cluster via the Harness Delegate.

Harness vs. ArgoCD — Role Clarity

Harness Responsibilities

- CI pipeline: build, test, scan
- Docker image building and pushing
- Pipeline governance and approvals
- Secret injection into pipelines
- Deployment strategy orchestration
- Triggering ArgoCD sync via API or manifest update
- Notifications, audit trail, RBAC

ArgoCD Responsibilities

- Watch Git repository for manifest changes
- Reconcile cluster state with Git state
- Detect and report configuration drift
- Kubernetes-native sync and rollback
- Application health visualization

Overlap / Integration Points

- Harness can trigger ArgoCD sync steps
- Harness can monitor ArgoCD app health
- Both use the same Git repository as source of truth
- Delegate bridges Harness to ArgoCD's Kubernetes API

The Concept Map — Jenkins to Harness

Your Translation Dictionary

Use this as your reference when reading Harness documentation. Every Jenkins concept has an approximate Harness equivalent — but the architecture behind each concept may differ significantly.

Jenkins Concept	Harness Equivalent	Notes
Jenkins Controller	Harness Platform / SaaS	Different deployment model
Jenkins Agent	Harness Delegate	Conceptually similar, architecturally different
Jenkinsfile (Groovy DSL)	Harness Pipeline (YAML)	Declarative, versioned in Git
Jenkins Credentials	Harness Secrets Manager	Supports Vault, AWS SM, etc.
Jenkins Plugins	Harness Connectors	Git, Docker, K8s, cloud connectors
Shared Library	Harness Template Library	Step, Stage, and Pipeline templates
Pipeline Stage	Harness Stage (CI / CD)	Type-specific stage definitions
Build Node Label	Delegate Selector / Tag	Route tasks to specific Delegates

Deployment Model Summary — Status Labels

Reference Guide for All Harness Deployment Options

When students or instructors ask "can we do this with Harness?", use the following status labels to give a precise and accurate answer.

SUPPORTED

- Harness SaaS (fully managed)
- Delegate in Kubernetes Pod
- Delegate in Docker container
- Harness Open Source via Docker
- SMP on Kubernetes via Helm

CONCEPTUALLY SIMILAR

- Delegate $\approx$ Jenkins Agent (role)
- Harness SMP $\approx$ Jenkins self-hosted (concept)
- Harness SaaS $\approx$ Managed Jenkins hosting
- Harness + ArgoCD $\approx$ Jenkins + ArgoCD

NOT SUPPORTED / NOT RECOMMENDED

- SMP as a single Docker container
- docker-compose for full SMP
- Treating Delegate as the Harness platform
- SMP co-located on student workload cluster

Key Takeaways — Module 8

What Every Student Must Leave This Module Knowing

1 Harness is not one thing

SaaS, Delegate, Self-Managed Enterprise, and Open Source are distinct products with different deployment models, requirements, and use cases.

2 The Delegate is not the platform

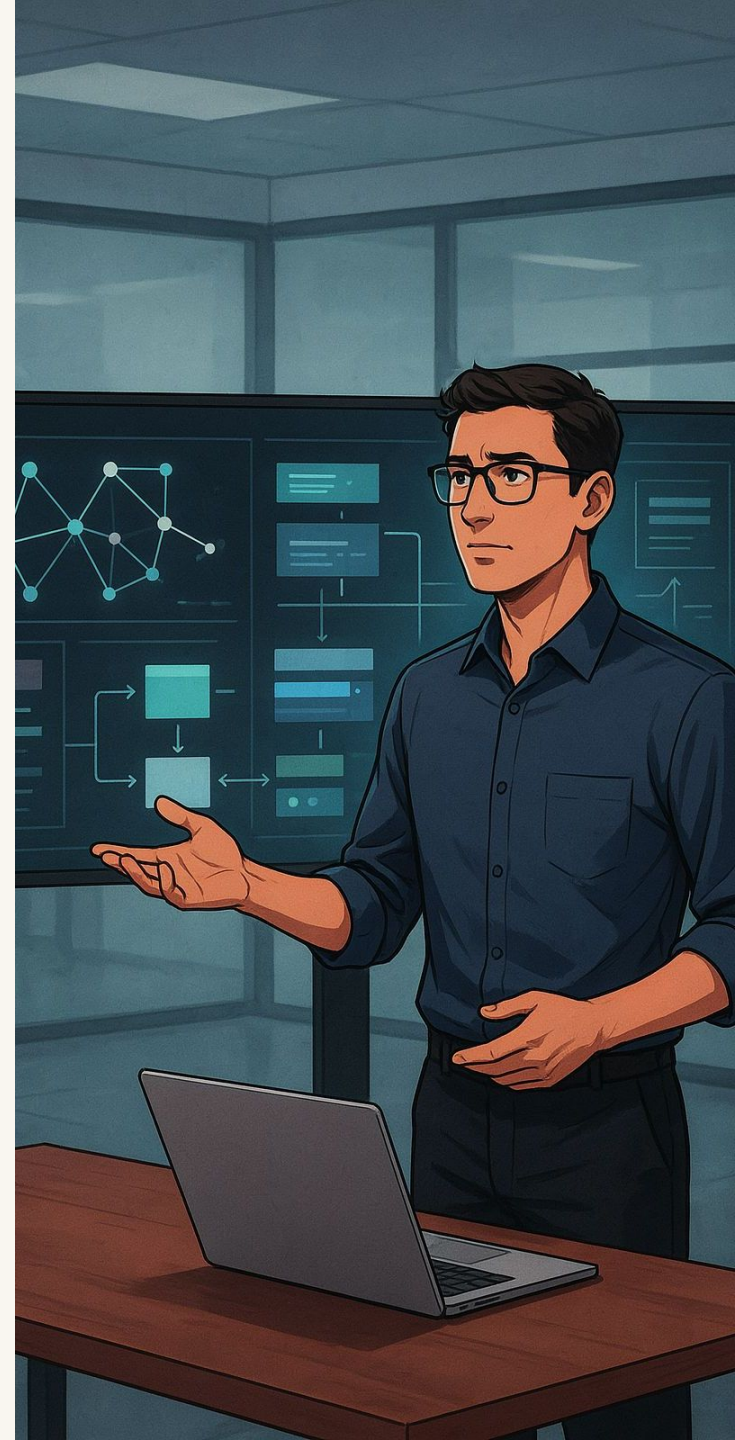
Running a Harness Delegate in Docker does not mean you are running the Harness platform. Delegate $\neq$ Harness Controller.

3 Self-Managed requires Kubernetes

Harness Self-Managed Enterprise Edition is a multi-service platform installed via Helm on Kubernetes. There is no single-container equivalent.

4 SaaS + Kubernetes Delegate is your training path

Use Harness SaaS with a Kubernetes Delegate in EKS Anywhere. This gives you all Harness learning opportunities without platform operational burden.



Final Warning — The #1 Misconception

Harness ≠ Jenkins in Another Container

What Jenkins Lets You Do

docker run jenkins/jenkins
One container. Full Controller. UI + orchestration + credentials. Everything in one process.

What Harness Self-Managed Is

A multi-service Kubernetes platform with dozens of microservices, databases, and supporting components — installed via Helm. Not a single container.

The Right Harness Equivalent

For learning: use **Harness SaaS** — Harness hosts the platform for you. For self-hosted: use **Harness SMP on Kubernetes** with dedicated infrastructure.

- ❌ If a student asks "can I just docker run the Harness controller?" — the answer is: that is not how Harness Self-Managed Enterprise Edition works. Direct them to Harness SaaS for learning, or Kubernetes + Helm for self-managed deployment.